

OpenCode + Codex + tmux + Herdr

科研服务器新手教程 v2 • curl 安装 + SSH 隧道认证版

适用于 Linux 科研服务器（Bash + Conda 环境）。推荐使用独立 agent 环境管理 Node.js、npm、OpenCode 和 Codex。

目标：先判断服务器网络与依赖条件，再选择 **curl / npm / Conda 隔离环境 / 手工二进制** 中最合适的安装路线；必要时通过 Windows Clash + SSH 反向隧道让服务器稳定联网，并用 tmux 或 Herdr 保持长任务。

0. 服务器基线

你当前的 Conda

```
conda env list

# conda environments:
#
base                  * /home/gtang/miniconda3
agent                 /home/gtang/miniconda3/envs/agent
```

建议：**Node.js、OpenCode、Codex 都装在 agent 环境**；base 保持干净。

整体结构

```
SSH / Xshell
├── tmux 或 Herdr      ← 保持会话
├── OpenCode / Codex  ← AI coding agent
│   └── Node/npm 位于 agent 环境
```

1. 安装前检查：先判断网络、系统和依赖

推荐顺序：先检查 curl 是否能快速访问；再检查 Node/npm、ripgrep 和系统架构；根据结果选择后面的安装路线。不要一上来就反复重装。

1.1 基础环境检查

```
uname -s
uname -m
ldd --version | head -n 1

command -v curl
command -v tar
command -v gzip
command -v git
command -v rg
command -v node
command -v npm
```

对常见科研服务器，OpenCode Linux x86_64 二进制通常对应 x86_64。如果是老 CPU，还建议检查 AVX2：

```
grep -m1 -o 'avx2' /proc/cpuinfo || echo "No AVX2 detected"
```

如果没有 AVX2，手工下载二进制时优先选择带 baseline 的 x64 构建；ARM64 服务器则选择 linux-arm64。

1.2 先测试 curl：能快就走安装脚本，慢就别硬等

先做一个 15 秒以内的连通性测试：

```
curl -I -L --connect-timeout 5 --max-time 15 https://opencode.ai/install
```

如果很快返回 HTTP 状态码，可优先走 **方案 A：curl 官方安装脚本**。如果长时间卡住、超时或速度很慢，直接转到 npm / Conda / SSH 隧道方案。

1.3 检查 Node/npm：决定能否用 npm 安装

```
node -v
npm -v
npm config get prefix
npm view opencode-ai version
```

结果	建议
Node/npm 已存在, npm registry 可访问, global prefix 可写	直接使用方案 B1: 现有 npm 安装
Node/npm 不存在, 或系统 npm 没有写权限	使用方案 B2: Conda 单独建立 agent 环境
npm registry 也无法访问	使用方案 C: Windows 下载 Linux 可执行文件 + SCP 上传

1.4 检查 ripgrep: 离线环境尤其建议提前装

OpenCode 会使用 rg (ripgrep) 进行文件搜索。没有外网时, 程序可能尝试后台下载而失败, 因此建议预先准备:

```
rg --version

# 如果没有 rg, 而 Conda 可用:
conda install -n agent -c conda-forge ripgrep -y
```

1.5 推荐: 让 base 也能直接调用 opencode / codex

如果软件实际安装在 agent 环境, 可只给命令做软链接, 不要把整个 agent/bin 永久放到 PATH 最前面:

```
mkdir -p ~/.local/bin

grep -qxF 'export PATH="$HOME/.local/bin:$PATH"' ~/.bashrc || \
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc

source ~/.bashrc
```

具体的 opencode / codex 软链接在各自安装完成后再创建。

2. SSH 隧道代理：通过 Windows Clash 解决服务器网络限制

部分科研服务器存在 GitHub、npm 或 AI 服务访问限制，可以通过 Windows 本机 Clash + SSH 隧道实现服务器联网。

2.1 Windows SSH 配置

```
Host your-server-tunnel
  HostName your.server.address
  User username
  Port 2222

  ServerAliveInterval 30
  ServerAliveCountMax 3
  ExitOnForwardFailure yes

# Clash 代理转发
RemoteForward 127.0.0.1:13130 127.0.0.1:7897
```

Windows 启动：

```
ssh -N your-server-tunnel
```

2.2 服务器端代理快捷设置

在服务器 ~/.bashrc 中加入：

```
# =====
# Windows Clash SSH tunnel proxy
# =====

proxy_on() {

  local PORT=13130

  if ! timeout 1 bash -c "</dev/tcp/127.0.0.1/$PORT" 2>/dev/null; then
    echo "ERROR: SSH proxy tunnel 127.0.0.1:$PORT is not running."
    echo ""
    echo "Start on Windows:"
    echo "  ssh -N your-server-tunnel"
    return 1
  fi
```

```
export http_proxy=http://127.0.0.1:$PORT
export https_proxy=http://127.0.0.1:$PORT
export ftp_proxy=http://127.0.0.1:$PORT

export HTTP_PROXY=$http_proxy
export HTTPS_PROXY=$https_proxy
export FTP_PROXY=$ftp_proxy

export NO_PROXY="localhost,127.0.0.1,::1"
export no_proxy=$NO_PROXY

echo "Proxy ON → Windows Clash tunnel (127.0.0.1:$PORT)"
}

proxy_off() {

    unset http_proxy https_proxy ftp_proxy
    unset HTTP_PROXY HTTPS_PROXY FTP_PROXY
    unset NO_PROXY no_proxy

    echo "Proxy OFF"
}

proxy_status() {

    env | grep -i proxy

}

# =====
```

使用:

```
source ~/.bashrc

proxy_on
proxy_status

# 关闭
proxy_off
```

2. 安装 OpenCode: 四种情况按条件选择

不要把所有方法都执行一遍。按下面的判断表选择一条最合适的路线即可。你当前服务器已验证过：官方 HTTP 代理能被 curl 使用，但 OpenCode/Bun 可能出现 `Malformed_HTTP_Response`，这时 Windows Clash + SSH 隧道是很实用的备用网络路线。

服务器情况	推荐路线
<code>curl https://opencode.ai/install</code> 很快	A. curl 官方安装脚本
<code>curl</code> 慢，但已有 Node/npm，npm registry 正常	B1. 直接 npm 安装
没有 Node/npm，或想保持系统环境干净	B2. Conda 建独立 agent 环境后 npm 安装
服务器几乎不能联网 / GitHub、npm 都不通	C. Windows 下载 Linux 二进制 → SCP 上传 → SSH 隧道供运行时联网

2.1 方案 A：curl 官方安装脚本

官方最简命令：

```
curl -fsSL https://opencode.ai/install | bash
```

科研服务器网络不稳定时，更推荐先下载脚本再执行：

```
curl -fL \  
  --connect-timeout 10 \  
  --max-time 120 \  
  https://opencode.ai/install \  
  -o /tmp/opencode-install.sh  
  
head -n 10 /tmp/opencode-install.sh  
bash /tmp/opencode-install.sh  
  
opencode --version  
command -v opencode
```

如果这里下载几十秒都没有进展，不要一直等。直接切换到方案 B 或 C。

2.2 方案 B1：服务器已有 Node/npm，直接 npm 安装

```
node -v
npm -v
npm config get prefix
npm view opencode-ai version
```

如果 npm 工作正常，且 global prefix 对当前用户可写：

```
npm install -g opencode-ai@latest

hash -r
opencode --version
command -v opencode
```

如果出现 EACCES / Permission denied，尤其是 prefix 指向 /usr 或 /usr/local，不建议在共享科研服务器上直接 `sudo npm -g`；转到下面的 Conda 独立环境更稳。

2.3 方案 B2：Conda 建立独立 agent 环境，再用 npm 安装

如果 agent 环境还不存在：

```
conda create -n agent -c conda-forge nodejs=22 ripgrep -y
conda activate agent
```

如果 agent 环境已经存在：

```
conda install -n agent -c conda-forge nodejs=22 ripgrep -y
conda activate agent
```

然后：

```
node -v
npm -v
rg --version

npm install -g opencode-ai@latest
opencode --version
which opencode
```

让 base 环境也能直接启动：

```
mkdir -p ~/.local/bin
ln -sf "$CONDA_PREFIX/bin/opencode" ~/.local/bin/opencode

grep -qxF 'export PATH="$HOME/.local/bin:$PATH"' ~/.bashrc || \
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc

source ~/.bashrc
command -v opencode
```

2.4 方案 C：服务器不能联网——Windows 下载二进制 + SCP 上传

适合：服务器访问 GitHub/npm 很慢甚至完全不可用，但 Windows 本机网络正常。

2.4.1 服务器先确认架构

```
uname -m
grep -m1 -o 'avx2' /proc/cpuinfo || echo "No AVX2 detected"
```

服务器	Release 文件
x86_64 + 支持 AVX2	opencode-linux-x64.tar.gz
x86_64 + 老 CPU / 无 AVX2	opencode-linux-x64-baseline.tar.gz
ARM64 / aarch64	opencode-linux-arm64.tar.gz

Windows 浏览器打开官方 Releases: [GitHub Releases](#), 下载对应 Linux 压缩包。

2.4.2 Windows 先配置 SSH 简写

```
notepad $env:USERPROFILE\.ssh\config
```

加入：

```
Host bsc-t6
  HostName ssh.cn-zhongwei-1.paracloud.com
  User sch2887@BSCC-T6
  Port 2222
  ServerAliveInterval 30
  ServerAliveCountMax 3
```



```
Host bsc-t6-tunnel
  HostName ssh.cn-zhongwei-1.paracloud.com
  User sch2887@BSCC-T6
  Port 2222
  ServerAliveInterval 30
  ServerAliveCountMax 3
  ExitOnForwardFailure yes
  RemoteForward 127.0.0.1:13128 127.0.0.1:7897
```

以后正常登录：

```
ssh bsc-t6
```

2.4.3 把下载好的 OpenCode 上传到服务器

例如 Windows 下载文件位于 C:\Users\gangt\Downloads\opencode-linux-x64.tar.gz：

```
scp C:\Users\gangt\Downloads\opencode-linux-x64.tar.gz bsc-t6:~/
```

服务器上：

```
mkdir -p ~/software/opencode ~/.local/bin
cd ~/software/opencode

tar -xzf ~/opencode-linux-x64.tar.gz
chmod +x opencode
ln -sf "$PWD/opencode" ~/.local/bin/opencode

grep -qxF 'export PATH="$HOME/.local/bin:$PATH"' ~/.bashrc || \
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc

source ~/.bashrc
opencode --version
```

2.5 Windows Clash + SSH 反向隧道：让 T6 走本机网络

当前 Clash Verge 端口为 **7897**。Windows 先验证：

```
curl.exe -x http://127.0.0.1:7897 https://opencode.ai -I
```

正常后启动隧道：

```
ssh -N bsccl-t6-tunnel
```

窗口一直停在那里是正常的，不要关闭。链路：

```
T6 127.0.0.1:13128
  ↓
SSH RemoteForward
  ↓
Windows 127.0.0.1:7897
  ↓
Clash Verge
  ↓
Internet
```

另开窗口：

```
ssh bsccl-t6
```

T6 检查：

```
ss -lnt | grep 13128
curl -x http://127.0.0.1:13128 https://opencode.ai -I
```

2.6 在 T6 的 ~/.bashrc 中加入代理快捷命令

原来自动设置 10.255.128.100:3128 的三行 export
http_proxy/https_proxy/ftp_proxy 建议先注释掉，避免每次登录自动覆盖。

```
# ===== Proxy settings =====
proxy_on() {
    if ! timeout 1 bash -c '</dev/tcp/127.0.0.1/13128' 2>/dev/null; then
        echo "ERROR: SSH tunnel 127.0.0.1:13128 is not running."
        echo "Start it on Windows: ssh -N bsccl-t6-tunnel"
        return 1
    fi
    export http_proxy=http://127.0.0.1:13128
    export https_proxy=http://127.0.0.1:13128
    export ftp_proxy=http://127.0.0.1:13128
    export HTTP_PROXY=$http_proxy
    export HTTPS_PROXY=$https_proxy
    export NO_PROXY=localhost,127.0.0.1,::1
    export no_proxy=$NO_PROXY
```

```
    echo "Proxy → Windows Clash (127.0.0.1:13128)"
}

proxy_t6() {
    export http_proxy=http://10.255.128.100:3128
    export https_proxy=http://10.255.128.100:3128
    export ftp_proxy=http://10.255.128.100:3128
    export HTTP_PROXY=$http_proxy
    export HTTPS_PROXY=$https_proxy
    export NO_PROXY=localhost,127.0.0.1,::1
    export no_proxy=$NO_PROXY
    echo "Proxy → T6 official proxy (10.255.128.100:3128)"
}

proxy_off() {
    unset http_proxy https_proxy ftp_proxy
    unset HTTP_PROXY HTTPS_PROXY
    unset NO_PROXY no_proxy
    echo "Proxy OFF"
}

proxy_status() {
    env | grep -i proxy
}
```

保存后：

```
source ~/.bashrc
```

以后：

```
proxy_on      # Windows Clash
proxy_t6      # T6 官方代理
proxy_status  # 查看
proxy_off     # 关闭
```

2.7 隧道开启后，也可以重新使用 curl / npm 安装

```
proxy_on
curl -I https://opencode.ai
npm view opencode-ai version

# 二选一
curl -fsSL https://opencode.ai/install | bash
# 或
npm install -g opencode-ai@latest
```

2.8 安装完成后的最小验证

```
opencode --version
opencode models | head
opencode run --pure --model opencode-go/grok-4.6 "Reply with exactly OK"
```

如果 T6 官方代理下出现 `Malformed_HTTP_Response`，而 Windows 隧道下正常：

```
# Windows
ssh -N bsc-t6-tunnel

# T6
proxy_on
opencode
```

2.9 OpenCode V2（可选）

```
# V2 npm
npm install -g @opencode/cli

# V2 curl
curl -fsSL https://opencode.ai/v2/install | bash
```

V1 → V2 的配置字段和权限规则存在变化。当前服务器已经稳定运行 V1 时，不需要为了安装方式强制升级。

2.10 连接 OpenCode Go

```
opencode
```

进入 TUI 后：

```
/connect
/models
```

3. OpenCode 常用操作

3.1 最常用斜杠命令

命令	用途
/connect	连接模型提供商，例如 OpenCode Go
/models	切换模型
/themes	切换 TUI 主题
/new	新会话；别名 /clear
/sessions	查看/恢复历史会话；别名 /resume、/continue
/compact	压缩长会话上下文；别名 /summarize
/init	创建/更新 AGENTS.md
/details	显示/隐藏工具执行细节
/thinking	切换“思考块”显示；不等于切换模型推理能力
/undo	撤销最后一轮及相应文件修改
/redo	恢复刚撤销的修改
/export	导出当前会话
/help	查看帮助
/exit	退出；别名 /quit、/q

3.2 文件和 Shell 快捷方式

```
@filename.py      # 在提示词中引用文件
!ls -la          # 直接运行 shell 命令
!git status
```

3.3 启动时指定模型

```
opencode -m provider/model
```

```
# 或继续上次会话
opencode -c
```

3.4 Auto / Permission (V1)

临时自动批准：

```
opencode --auto
```

永久全部允许：

```
mkdir -p ~/.config/opencode

cat > ~/.config/opencode/opencode.json <<'EOF'
{
  "$schema": "https://opencode.ai/config.json",
  "permission": "allow"
}
EOF
```

更保守的示例：

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "*": "ask",
    "bash": "allow",
    "edit": "allow"
  }
}
```

V1 中审批结果通常有：**once**（只允许一次）、**always**（本会话/匹配规则持续允许）、**reject**（拒绝）。

► **OpenCode V2 的 full allow 写法**

4. 安装 Codex CLI

如果服务器无法稳定访问 npm / OpenAI, 可先在 Windows 执行 `ssh -N bsc-t6-tunnel`, 再在 T6 执行 `proxy_on`; 后面的 npm 安装和登录都会继承该代理环境。

4.1 推荐安装方式: curl 官方安装脚本

新版 Codex CLI 推荐优先使用官方安装脚本:

```
curl -fsSL https://openai.com/codex/install.sh | sh

codex --version
which codex
```

4.2 Conda agent 环境安装 (适合科研服务器)

Codex CLI 通过 npm 安装, 依赖 agent 环境中的 Node.js, 因此建议保持在 agent 环境中运行:

```
conda activate agent

npm install -g @openai/codex

codex --version
which codex
```

4.3 地域限制与 SSH 隧道认证方案

部分服务器出口网络可能导致 Codex 登录阶段出现地区限制。推荐通过 Windows 本机 Clash + SSH 隧道完成认证。

Windows SSH 配置增加 LocalForward (仅登录认证阶段使用):

```
Host bsc-t6-tunnel
  HostName ssh.cn-zhongwei-1.paracloud.com
  User sch2887@BSCC-T6
  Port 2222
  ServerAliveInterval 30
  ServerAliveCountMax 3
  ExitOnForwardFailure yes

  RemoteForward 127.0.0.1:13128 127.0.0.1:7897

  # Codex 登录认证临时开启
  LocalForward 127.0.0.1:1455 127.0.0.1:1455
```

Windows 启动:

```
ssh -N bsccl-t6-tunnel
```

服务器设置代理:

```
export http_proxy=http://127.0.0.1:13128  
export https_proxy=http://127.0.0.1:13128
```

然后执行:

```
codex login
```

浏览器认证完成后, 关闭 SSH 隧道, 删除或注释:

```
# LocalForward 127.0.0.1:1455 127.0.0.1:1455
```

后续正常使用只需要保留 RemoteForward 代理。

4.4 登录

```
codex login
```

检查状态:

```
codex login status
```

退出:

```
codex logout
```

如果出现地区限制提示, 优先检查 SSH 隧道出口和认证转发, 不要反复重装 Codex。

5. Codex 常用操作

5.1 进入项目

```
cd ~/your-project
codex
```

5.2 最常用斜杠命令

命令	用途
/model	切换模型
/reasoning	切换推理强度
/permissions	切换权限/审批预设，例如 Auto、Read Only
/theme	切换语法高亮主题并写入配置
/status	查看模型、审批策略、可写目录、token 使用等
/compact	压缩上下文
/new	新建聊天
/fork	从当前聊天分叉
/review	代码审查
/diff	查看 Git diff
/copy	复制最新完成的 Codex 输出
/init	生成 AGENTS.md 模板
/mcp	查看 MCP 状态
/plan	切换计划模式
/personality	切换响应风格（模型支持时）
/exit / /quit	退出 CLI

5.3 Auto / 权限

最推荐：进入 Codex 后直接：

```
/permissions
```

然后选 **Auto**（更自动）或 **Read Only**（更保守）。

命令行低打扰运行：

```
codex --sandbox workspace-write --ask-for-approval on-request
```

如果你明确希望“不询问”，但仍保留沙箱：

```
codex --sandbox workspace-write --ask-for-approval never
```

共享科研服务器不建议： `codex --yolo / --dangerously-bypass-approvals-and-sandbox` 会同时绕过审批和沙箱。只应在你自己额外隔离的临时环境里使用。

5.4 常用快捷输入

@file.py	# 引用工作区文件
!git status	# 在当前 <code>sandbox/approval</code> 设置下执行 <code>shell</code>
Ctrl+O	# 复制最新完成的 <code>Codex</code> 输出
Ctrl+C	# 结束当前 <code>Codex</code> 会话

5.5 Codex 自动审批（YOLO 模式）与长期任务

YOLO 自动批准模式

为了减少科研开发过程中的频繁确认，可以设置快捷命令：

```
# ~/.bashrc

function cdx(){

    if ! env | grep -q "http_proxy"; then
        echo "Proxy not enabled. Run proxy_on first."
    fi

    codex --yolo "$@"
}
```

重新加载：

```
source ~/.bashrc
```

以后：

```
cd ~/project  
cdx
```

Codex 长期任务 / Goal 模式

对于较大的科研任务，建议使用目标驱动方式：

```
/goal
```

适合：

- 持续修改大型 Python 工程
- VASP / LAMMPS 工作流自动化
- 机器学习数据处理 pipeline
- 多步骤科研代码开发

6. tmux：最稳的 SSH 断线保护

6.1 安装

你没有 sudo 也没关系，可以直接装到 base Conda：

```
conda activate base  
conda install -c conda-forge tmux -y  
  
tmux -V
```

6.2 最常用命令

```
# 新建会话  
tmux new -s agent  
  
# 查看会话
```

```
tmux ls

# 重新进入
tmux attach -t agent

# 结束整个会话
tmux kill-session -t agent
```

6.3 tmux 常用按键

tmux 默认前缀是：先按 **Ctrl** + **B**，松开，再按后面的键。

操作	按键
Detach，后台继续	Ctrl+B → D
新窗口	Ctrl+B → C
左右分屏	Ctrl+B → %
上下分屏	Ctrl+B → "
切换 pane	Ctrl+B → 方向键
关闭当前 pane	Ctrl+B → X
滚屏/复制模式	Ctrl+B → [

6.4 典型用法

```
tmux new -s opencode

cd ~/your-project
opencode
```

要关 Xshell： **Ctrl+B** → **D**。第二天：

```
tmux attach -t opencode
```

7. Herdr： 更适合多 Agent 的现代方案

7.1 安装

```
curl -fsSL https://herdr.dev/install.sh | sh

source ~/.bashrc
herdr --version
```

如果服务器访问 GitHub Release 超时，可以在本机下载 Linux x86_64 二进制，再用 scp 上传到 ~/.local/bin/herdr。

7.2 启动

```
cd ~/your-project
herdr
```

进入 Herdr pane 后：

```
opencode
# 或
codex
```

7.3 最常用 Herdr 按键

Herdr 默认前缀同样是 `Ctrl+B`。

操作	按键
Detach，后台继续	<code>Ctrl+B</code> → <code>Q</code>
新 Tab	<code>Ctrl+B</code> → <code>C</code>
向右分屏	<code>Ctrl+B</code> → <code>V</code>
向下分屏	<code>Ctrl+B</code> → <code>-</code>
pane 间移动	<code>Ctrl+B</code> → <code>H/J/K/L</code>
最大化/恢复 pane	<code>Ctrl+B</code> → <code>Z</code>
关闭 pane	<code>Ctrl+B</code> → <code>X</code>

下/上一个 Tab	Ctrl+B → N/P
Workspace 导航	Ctrl+B → W
查看全部快捷键	Ctrl+B → ?

7.4 退出和重连

只退出客户端但不停止任务：

```
# 在 Herdr 中：  
Ctrl+B → Q
```

重新连接：

```
herdr
```

真正停止 Herdr 会话和其中 pane：

```
herdr server stop
```

8. 推荐 workflow

方案 A：服务器网络正常

```
ssh bscct6  
  
tmux new -s ai  
cd ~/project  
opencode
```

适合：服务器可直接联网或 T6 官方代理能满足当前工具。

方案 B：OpenCode 走 Windows Clash

```
# Windows 窗口 1
ssh -N bsc-t6-tunnel

# Windows 窗口 2
ssh bsc-t6

# T6
proxy_on
tmux new -s ai
cd ~/project
opencode
```

适合：官方代理下 OpenCode/Bun 报 HTTP 异常，或希望统一走本机 Clash 网络。

方案 C：多 Agent

```
# 如需本地代理，先 proxy_on
cd ~/project
herdr

# pane 1
opencode
# pane 2
codex
```

适合：同时跑多个 Agent、多个 pane。

日常最短记忆

```
# Windows
ssh -N bsc-t6-tunnel

# T6
proxy_on
opencode
```

结束后可执行 proxy_off。

推荐：tmux 作为基础设施长期保留；Herdr 按需用于多 Agent。OpenCode / Codex 优先安装到独立 agent 环境；网络不稳定时使用 Windows Clash + SSH 反向隧道。

9. 常见问题

▼ Q1: base 下 opencode / codex 找不到

```
ls -l ~/miniconda3/envs/agent/bin/opencode
ls -l ~/miniconda3/envs/agent/bin/codex

mkdir -p ~/.local/bin
ln -sf ~/miniconda3/envs/agent/bin/opencode ~/.local/bin/opencode
ln -sf ~/miniconda3/envs/agent/bin/codex ~/.local/bin/codex

export PATH="$HOME/.local/bin:$PATH"
```

► Q2: OpenCode 想切模型

► Q3: OpenCode 想换主题

► Q4: Codex 想换模型、推理强度、主题

► Q5: Codex / OpenCode 总是问权限

► Q6: SSH / Xshell 断线了怎么办

► Q7: curl 能访问 opencode.ai, 但 OpenCode 报 Malformed_HTTP_Response

► Q8: 怎么确认 SSH 隧道还活着

官方资料

本教程按 2026-09-16 的官方文档核对。产品更新很快，遇到命令不一致时以官方文档和本机 `--help` 为准。

[OpenCode](#)[OpenCode Docs](#)[OpenCode V2](#)[V2 Docs](#)[OpenCode Releases](#)[GitHub Releases / Linux binaries](#)[Codex](#)[Codex CLI](#)[Codex commands](#)[Codex Slash Commands](#)[Herdr](#)[Herdr Docs](#)[tmux](#)

本页使用标准默认键位。

适用环境: Rocky Linux / Bash / Conda / Windows PowerShell + Clash Verge · 示例 SSH: BSCC-T6 · 隧道端口: 13128
→ Windows 7897